

LogiFlow — Week 1 Progress Report

Candidate: Aryan Ajmera

Reference Number: NFGT/HR/INT/2026/160

Week: 1 of 8 (Backend Engineering & REST APIs — Weeks 1–4)

Date: July 29, 2026

1. Tasks Completed

Task W1.1 — Project Setup, Architecture & MongoDB Schema Design

- Initialized the Node.js/Express backend with full MVC folder structure (routes/, controllers/, models/, middleware/, config/, utils/)
- Connected the application to MongoDB Atlas using Mongoose
- Designed and implemented 13 Mongoose schemas: User, Warehouse, Product, Inventory, Customer, Order, Shipment, Vehicle, Driver, Dispatch, Ticket, Notification, and ActivityLog
- Used ObjectId references between related models, and enums/required validators for data integrity
- Implemented OrderItem and ShipmentEvent as subdocument arrays (nested in Order and Shipment) rather than separate collections, since they only exist in the context of their parent
- Implemented Role as an enum field on User rather than a standalone model, since roles are a small fixed set of values
- Configured dotenv for environment variables
- Initialized Git repository with main and develop branches, and added a .gitignore excluding node_modules and .env
- Documented the full schema design in README.md

Task W1.2 — JWT Authentication System & RBAC Middleware

- Implemented POST /api/auth/register with bcrypt password hashing (saltRounds=12)
- Implemented POST /api/auth/login with credential verification and JWT issuance (8-hour expiry)
- Implemented POST /api/auth/logout
- Built verifyToken middleware to extract and verify the Bearer token, attaching the decoded payload to req.user
- Built authorizeRoles(...allowedRoles) middleware to restrict access based on role
- Created rolePermissions.js as a foundation for module-level permission mapping in later weeks
- Built and tested two protected routes to validate the middleware: GET /api/auth/me (any authenticated user) and GET /api/auth/users (Administrator-only)
- Confirmed in Postman that a non-Administrator role correctly receives a 403 Forbidden on the admin-only route

Task W1.3 — Week 1 Integration & Review

- Verified verifyToken and authorizeRoles behave correctly across 2 protected routes covering 2 different roles
- Registered and inserted 5 test users into the database via Postman, one for each of the 5 roles
- Verified Mongoose schema validators reject invalid documents: missing required field, invalid role outside the enum, and duplicate email — all correctly rejected
- Committed all work to the develop branch
- Opened and merged a GitHub Pull Request from develop into main for Week 1

2. Blockers Faced & Resolutions

- MongoDB Atlas authentication failure: "bad auth" error traced to special characters in the database password requiring URL encoding. Resolved by regenerating an autogenerated, URL-safe password via Atlas.
- Port conflict with macOS AirPlay Receiver: server returned 403 Forbidden with no server-side logs. Diagnosed via response headers (Server: AirTunes) that AirPlay Receiver occupies port 5000 by default. Resolved by moving the backend to port 8000.
- Missing files and dependency gaps: config/db.js and routes/authRoutes.js were referenced before being created, and bcryptjs was referenced before installation, producing MODULE_NOT_FOUND errors. Resolved by systematically checking file existence and reconciling package.json against required dependencies.
- Postman VS Code extension instability: request-builder UI failed to render reliably. Worked around with curl for interim testing, then installed Postman Desktop, which resolved it completely.

3. What Was Learned

- Structuring a backend using the MVC pattern, and why separating routing from business logic keeps the codebase maintainable
- Designing MongoDB/Mongoose schemas with references (ObjectId + ref) versus subdocuments, and when to use each
- Implementing secure password storage with bcrypt hashing
- JWT-based stateless authentication: signing tokens with a payload and expiry, verifying via middleware instead of server-side sessions
- Building composable Express middleware, including the "function that returns a middleware function" pattern used for authorizeRoles
- Practical debugging: diagnosing environment-level issues (port conflicts, missing files, credential formatting) using terminal tools, response headers, and systematic elimination

4. Plan for Next Week (Week 2)

- Task W2.1: Warehouse & Product Management APIs — full CRUD endpoints with role-based restrictions
- Task W2.2: Inventory Management APIs — stock tracking, low-stock alerting, and safeguards against negative inventory
- Task W2.3: Week 2 integration testing and review